

Introducción al ciclo de vida de desarrollo de software móvil

Consideraciones al desarrollar aplicaciones móviles

Información general

La creación de aplicaciones móviles puede ser tan fácil como abrir el IDE, juntar algunos elementos, realizar unas pruebas rápidas y enviar el resultado al App Store, todo ello en una misma tarde. O puede ser un proceso muy complicado que conlleva el diseño riguroso por adelantado, pruebas de facilidad de uso, pruebas de control de calidad en miles de dispositivos, un ciclo de vida completo de versión beta y la implementación de varias maneras diferentes.

En este documento, vamos a realizar un examen de introducción exhaustivo de la creación de aplicaciones móviles, incluidos:

1. **Proceso:** el proceso de desarrollo de software se denomina ciclo de vida de desarrollo de software (SDLC). Examinaremos todas las fases del SDLC con respecto al desarrollo de aplicaciones móviles, incluidas: inspiración, diseño, desarrollo, estabilización, implementación y mantenimiento.
2. **Consideraciones:** hay una serie de consideraciones que se deben tener en cuenta al crear aplicaciones móviles, especialmente si se comparan con las aplicaciones de escritorio o web tradicionales. Examinaremos estas consideraciones y cómo afectan al desarrollo móvil.

Este documento está pensado para responder a preguntas fundamentales sobre el desarrollo de aplicaciones móviles, para desarrolladores de aplicaciones nuevos y experimentados. Adopta un enfoque bastante completo para introducir la mayor parte de los conceptos que se ejecutarán durante todo el ciclo de vida de desarrollo de software (SDLC).

En cambio, puede que este documento no sea para todos los usuarios, si quiere empezar a crear aplicaciones, le recomendamos que pase a los tutoriales [Introducción al desarrollo móvil](#), [Hello, Android](#) o [Hello, iPhone](#) y vuelva a este documento más adelante.

SDLC de desarrollo móvil

El ciclo de vida de desarrollo móvil es, en gran medida, parecido al SDLC para aplicaciones web o de escritorio. Al igual que con estos, normalmente hay 5 partes importantes del proceso:

1. **Inicio:** todas las aplicaciones se inician con una idea. Normalmente, la idea se perfecciona en una base sólida para una aplicación.
2. **Diseño:** la fase de diseño consiste en definir la experiencia del usuario (UX) de la aplicación (como cuál es el diseño general, cómo funciona, etc.), así como convertir esa experiencia del usuario en un diseño de interfaz de usuario (UI) adecuado, normalmente con la ayuda de un diseñador gráfico.
3. **Desarrollo:** normalmente, es la fase con un uso más intensivo de recursos, esta es la creación real de la aplicación.
4. **Estabilización:** cuando el desarrollo ha avanzado lo suficiente, normalmente el control de calidad empieza a probar la aplicación y se corrigen los errores. A veces, una aplicación pasará a una fase beta limitada en la que una audiencia de usuarios más amplia tiene la oportunidad de usarla, enviar comentarios y notificar cambios.
5. **Implementación**

A menudo, muchas de estas partes se superponen, por ejemplo, es común que el desarrollo siga mientras se finaliza la interfaz de usuario e incluso puede afectar al diseño de la interfaz de usuario. Además, una aplicación puede estar en una fase de estabilización al mismo tiempo que se agregan nuevas características a una nueva versión.

Asimismo, estas fases se pueden usar en varias metodologías de SDLC como Agile, Spiral, Waterfall, etc.

Cada una de estas fases se explicará con más detalle en las secciones siguientes.

Inicio

La ubicuidad y el nivel de interacción que tienen los usuarios con los dispositivos móviles implica que casi todos tienen una idea de una aplicación móvil. Los dispositivos móviles abren una forma totalmente nueva de interactuar con la informática, la web e incluso la infraestructura corporativa.

La fase de inicio consiste en definir y perfeccionar la idea de una aplicación. Para crear una aplicación correctamente, es importante hacerse algunas preguntas fundamentales. Aquí tiene algunas cuestiones que se deben tener en cuenta antes de publicar una aplicación en una de las App Store públicas:

- **Ventaja competitiva:** ¿ya existen aplicaciones similares? Si es así, ¿cómo se diferencia esta aplicación de las otras?

Para las aplicaciones que se distribuirán en una empresa:

- **Integración de la infraestructura:** ¿con qué infraestructura existente se integrará o cuál extenderá?

Además, las aplicaciones deben evaluarse en el contexto del factor de forma móvil:

- **Valor:** ¿qué valor aporta esta aplicación a los usuarios? ¿Cómo la usarán?
- **Forma/movilidad:** ¿cómo funcionará esta aplicación en un factor de forma móvil? ¿Cómo puedo agregar valor mediante tecnologías móviles, como el reconocimiento de ubicación, la cámara, etc.?

Para ayudarle a diseñar la funcionalidad de una aplicación, puede ser útil definir actores y [casos de uso](#). Los actores son roles dentro de una aplicación y, a menudo, son usuarios. Los casos de uso son, normalmente, acciones o intentos.

Por ejemplo, una aplicación de seguimiento de tareas puede tener dos actores: *usuario* y *amigo*. Un usuario podría *Crear una tarea* y *Compartir una tarea* con un amigo. En este caso, crear y compartir una tarea son dos casos de uso distintos que, junto con los actores, notificarán qué pantallas tiene que compilar, así como qué lógica y entidades empresariales se deben desarrollar.

Una vez que se ha capturado una cantidad apropiada de casos de uso y actores, es mucho más fácil empezar a diseñar una aplicación. El desarrollo se puede centrar entonces en cómo crear la aplicación, en lugar de qué es la aplicación o qué debería hacer.

Diseñar aplicaciones móviles

Una vez que se hayan determinado las características y funcionalidades de la aplicación, el siguiente paso es intentar resolver la experiencia del usuario o UX.

Diseño de la experiencia del usuario

Normalmente, la experiencia del usuario se realiza a través de tramas de alambres o prototipos con herramientas como [Balsamiq](#), [Mockingbird](#), [Visio](#) o simplemente el lápiz y papel de siempre. Los prototipos de experiencia del usuario permiten diseñar esta experiencia sin tener que preocuparse por el diseño real de la interfaz de usuario:



Al crear prototipos de experiencia del usuario, es importante tener en cuenta las instrucciones de la interfaz para las diferentes plataformas a las que se dirigirá la aplicación. La aplicación debería "sentirse cómoda" en todas las plataformas. Las instrucciones de diseño oficial para cada plataforma son:

1. **Apple:** [Directrices de interfaz humana](#)
2. **Android:** [Instrucciones de diseño](#)
3. **Windows Phone:** [Biblioteca de diseño para Windows Phone](#)

Por ejemplo, cada aplicación tiene una metáfora para cambiar de sección en una aplicación. iOS usa una barra de pestañas en la parte inferior de la pantalla, mientras que Android usa una barra de pestañas en la parte superior de la pantalla. Por otro lado, Windows Phone usa la vista Panorámica:



Además, el propio hardware también impone decisiones de la experiencia del usuario. Por

ejemplo, los dispositivos iOS no tienen ningún botón *Atrás* físico y, por tanto, presentan la metáfora de controlador de navegación:



Asimismo, el factor de forma también influye en las decisiones de la experiencia del usuario. Una tableta tiene mucho más espacio y, por tanto, puede mostrar más información. A menudo, lo que necesita varias pantallas en un teléfono, se comprime en una para una tableta:



Debido a la infinidad de factores de forma que existen, a menudo hay factores de forma medianos (entre un teléfono y una tableta) que puede que también quiera abordar.

Diseño de la interfaz de usuario (UI)

Una vez determinada la experiencia del usuario, el siguiente paso es crear el diseño de la interfaz de usuario. Mientras que la experiencia del usuario suele componerse de prototipos en blanco y negro, la fase de diseño de la interfaz de usuario es donde se introducen y finalizan los colores, gráficos, etc. Es importante dedicar tiempo a un buen diseño de la interfaz de usuario y, por lo general, las aplicaciones más populares tienen un diseño profesional.

Al igual que con la experiencia del usuario, es importante comprender que cada plataforma tiene su propio lenguaje de diseño, por lo que una aplicación bien diseñada puede tener un aspecto diferente en cada plataforma:



Para obtener inspiración sobre el diseño de la interfaz de usuario, consulte algunos de los siguientes sitios:

1. pttrns.com: (solo iOS)
2. androidpttrns.com: (solo Android)
3. lovelyui.com: (iOS, Android y Windows Phone)

4. mobiledesignpatterngallery.com: (iOS, Android y Windows Phone)

Además, es posible ver las carteras de diseñadores gráficos en sitios como Behance.com y Dribbble.com. Ahí se pueden encontrar diseñadores de todo el mundo, a menudo en lugares en que el tipo de cambio es favorable, por lo que un buen diseño gráfico no tiene que costar mucho necesariamente.

Desarrollo

Normalmente, la fase de desarrollo se inicia muy pronto. De hecho, una vez que una idea ha madurado un poco en la fase conceptual o de inspiración, a menudo se desarrolla un prototipo de trabajo que valida la funcionalidad, las suposiciones y ayuda a comprender el ámbito del trabajo.

En el resto de los tutoriales, nos centraremos principalmente en la fase de desarrollo.

Estabilización

La estabilización es el proceso de solucionar los errores de la aplicación. No solo desde un punto de vista funcional (por ejemplo: "Se bloquea al hacer clic en este botón"), sino también de facilidad de uso y rendimiento. Es mejor empezar la estabilización muy pronto en el proceso de desarrollo para que se puedan realizar correcciones antes de que sean costosas. Normalmente, las aplicaciones pasan por las fases *Prototipo*, *Alfa*, *Beta* y *Versión candidata para lanzamiento*. Hay personas que las definen de forma diferente, pero, generalmente, siguen el modelo siguiente:

1. **Prototipo**: la aplicación aún está en fase de prueba de concepto y solo funcionan la funcionalidad principal o determinadas partes de la aplicación. Hay errores principales.
2. **Alfa**: la funcionalidad principal tiene normalmente el código completado (compilado, pero no probado por completo). Aún hay errores principales, puede que no haya funcionalidades aisladas.
3. **Beta**: la mayoría de la funcionalidad está completa y ha pasado al menos una corrección de errores y una prueba ligeras. Puede que aún haya problemas conocidos

principales.

4. **Versión candidata para lanzamiento:** toda la funcionalidad está completa y probada. Salvo errores nuevos, la aplicación es una versión candidata para lanzamiento.

Nunca es demasiado pronto para empezar a probar una aplicación. Por ejemplo, si se encuentra un problema importante en la fase de prototipo, aún se puede modificar la experiencia del usuario de la aplicación para darle cabida. Si se encuentra un problema de rendimiento en la fase alfa, es lo suficientemente pronto para modificar la arquitectura antes de que se haya generado mucho código basado en suposiciones falsas.

Normalmente, según una aplicación avanza en el ciclo de vida, se abre a más personas para que la prueben, envíen comentarios, etc. Por ejemplo, las aplicaciones prototipo pueden mostrarse solo a partes interesadas clave o ponerse a disposición de estas partes, mientras que las aplicaciones de versión candidata para lanzamiento pueden distribuirse a los clientes que se hayan registrado para obtener acceso anticipado.

Para realizar primeras pruebas y la implementación en pocos dispositivos, normalmente es suficiente implementar directamente desde un equipo de desarrollo. En cambio, si se va ampliando el público, se puede volver más complicado rápidamente. Por tanto, hay una serie de opciones de implementación de pruebas que facilitan este proceso al permitirle invitar a usuarios a un grupo de pruebas, lanzar compilaciones en la web y proporcionar herramientas que permiten que los usuarios envíen comentarios.

Algunas de las más populares son:

1. **Testflight:** se trata de un producto de iOS que le permite distribuir aplicaciones para probarlas, así como recibir informes de errores e información de uso de los clientes. Se incluye como parte de iTunes Connect y no está disponible si forma parte de una suscripción al programa para desarrolladores empresariales de Apple.
2. **LaunchPad (launchpadapp.com):** diseñado para Android, este servicio es muy similar a TestFlight.
3. **Vessel (vessel.io):** un servicio para iOS y Android que le permite supervisar el uso, realizar un seguimiento de los clientes e incluso realizar pruebas A/B desde la aplicación.
4. **hockeyapp.com:** proporciona un servicio de pruebas para iOS, Android y Windows

Phone.

Distribución

Una vez que la aplicación se ha estabilizado, es el momento de publicarla. Hay una serie de opciones de distribución diferentes, dependiendo de la plataforma.

Las aplicaciones de Xamarin.iOS y Objective-C se distribuyen exactamente del mismo modo:

1. **Apple App Store:** el App Store de Apple es un repositorio de aplicaciones en línea disponible de forma global que está integrado en Mac OS X a través de iTunes. Sin duda, es el método de distribución de aplicaciones más popular y permite a los desarrolladores comercializar y distribuir sus aplicaciones en línea con muy poco esfuerzo.
2. **Implementación interna:** la implementación interna está pensada para la distribución interna de aplicaciones corporativas que no están disponibles de forma pública a través del App Store.
3. **Implementación ad hoc:** la implementación ad hoc está pensada principalmente para desarrollo y pruebas y le permite implementar en un número limitado de dispositivos aprovisionados correctamente. Al implementar en un dispositivo mediante Xcode o Visual Studio para Mac, se conoce como implementación ad hoc.

Android

Todas las aplicaciones de Android deben estar firmadas antes de distribuirlas. Los desarrolladores firman sus aplicaciones mediante su propio certificado protegido por una clave privada. Este certificado puede proporcionar una cadena de autenticidad que une a un desarrollador de aplicaciones con las aplicaciones que ha creado y publicado. Debe tenerse en cuenta que, mientras que un certificado de desarrollo para Android puede estar firmado por una entidad de certificación reconocida, la mayoría de los desarrolladores no optan por usar estos servicios y firman de forma automática sus certificados. El propósito principal de los certificados es diferenciar entre las distintas aplicaciones y desarrolladores. Android usa

esta información para ayudar con el cumplimiento de delegación de permisos entre las aplicaciones y componentes que se ejecutan en el sistema operativo Android.

A diferencia de otras plataformas móviles populares, Android adopta un enfoque muy abierto para la distribución de aplicaciones. Los dispositivos no están bloqueados para usar una única tienda de aplicaciones aprobada. En su lugar, cualquier usuario puede crear una tienda de aplicaciones y la mayoría de teléfonos Android permite que las aplicaciones se instalen desde estas tiendas de terceros.

Esto facilita a los desarrolladores un canal de distribución posiblemente más grande y aún más complejo para sus aplicaciones. [Google Play](#) es la tienda de aplicaciones oficial de Google, pero hay muchas otras. Algunas populares son:

1. [AppBrain](#)
2. [Tienda Apps de Amazon para Android](#)
3. [Handango](#)
4. [GetJar](#)

Windows Phone

Las aplicaciones de Windows Phone se distribuyen a los usuarios a través de la Tienda Microsoft. Los desarrolladores envían sus aplicaciones al [Centro de desarrollo de Windows Phone](#) para que las aprueben y después aparecen en la Tienda.

Microsoft proporciona instrucciones detalladas para [implementar aplicaciones de Windows Phone](#) durante el desarrollo.

Siga [estos pasos](#) para publicar aplicaciones para pruebas beta y publicarlas en la tienda. Los desarrolladores pueden enviar sus aplicaciones y después proporcionar un vínculo de instalación para los evaluadores, antes de que la aplicación se revise y publique.

Consideraciones sobre el desarrollo móvil

Aunque desarrollar aplicaciones móviles no es fundamentalmente diferente que el desarrollo

web o de escritorio tradicional en términos de proceso o arquitectura, existen algunas consideraciones que se deben tener en cuenta.

Consideraciones comunes

Multitarea

La multitarea (varias aplicaciones que se ejecutan a la vez) tiene dos retos importantes en un dispositivo móvil. En primer lugar, dado el espacio real en pantalla limitado, es difícil mostrar varias aplicaciones de forma simultánea. Por tanto, en los dispositivos móviles solo puede haber una aplicación en primer plano de cada vez. En segundo lugar, tener varias aplicaciones abiertas y realizando tareas puede agotar rápidamente la carga de la batería.

Cada plataforma trata la multitarea de forma diferente, lo que exploraremos en breve.

Factor de forma

Normalmente, los dispositivos móviles se dividen en dos categorías (teléfonos y tabletas) con unos cuantos dispositivos intermedios. El desarrollo para estos factores de forma es generalmente muy similar, en cambio, el diseño de aplicaciones para ellos puede ser muy diferente. Los teléfonos tienen un espacio de pantalla muy limitado y las tabletas, aunque son más grandes, siguen siendo dispositivos móviles con menos espacio de pantalla que la mayoría de los portátiles. Por este motivo, los controles de interfaz de usuario de la plataforma móvil se han diseñado específicamente para ser efectivos en factores de forma más pequeños.

Fragmentación de sistema operativo y dispositivo

Es importante tener en cuenta distintos dispositivos en todo el ciclo de vida de desarrollo de software:

1. **Planeación y conceptualización:** tenga en cuenta que el hardware y las características varían de un dispositivo a otro, puede que una aplicación que se basa en determinadas características no funcione correctamente en algunos dispositivos.

Por ejemplo, no todos los dispositivos tienen cámaras, por lo que, si está creando una aplicación de mensajería de vídeo, puede que algunos dispositivos reproduzcan vídeos, pero no los puedan grabar.

2. **Diseño:** al diseñar la experiencia del usuario (UX) de una aplicación, preste atención a las diferentes relaciones y tamaños de pantalla de los dispositivos. Además, al diseñar la interfaz de usuario (UI) de una aplicación, se deben tener en cuenta diferentes resoluciones de pantalla.
3. **Desarrollo:** al usar una característica del código, siempre se debe probar primero la presencia de esa característica. Por ejemplo, antes de usar una característica de dispositivo, como una cámara, confirme siempre primero que el sistema operativo tenga esa característica. Después, al inicializar el dispositivo o característica, asegúrese de solicitar la compatibilidad actual del sistema operativo sobre el dispositivo y después use esas opciones de configuración.
4. **Pruebas:** es muy importante probar la aplicación al principio y con frecuencia en dispositivos reales. Puede haber incluso dispositivos con las mismas especificaciones de hardware en que varíe mucho su comportamiento.

Recursos limitados

Los dispositivos móviles son cada vez mejores con el tiempo, pero siguen siendo dispositivos móviles con capacidades limitadas en comparación con los equipos de escritorio o portátiles. Por ejemplo, los desarrolladores de escritorio normalmente no se preocupan por las capacidades de memoria; están acostumbrados a tener memoria virtual y física en grandes cantidades, mientras que en los dispositivos móviles se puede consumir toda la memoria disponible rápidamente solo con cargar algunas imágenes de alta calidad.

Además, las aplicaciones de uso intensivo del procesador, como juegos o reconocimiento de texto, pueden influir realmente en la CPU móvil y afectar negativamente al rendimiento del dispositivo.

Debido a consideraciones como estas, es importante crear el código de manera inteligente y realizar la implementación pronto y con frecuencia en dispositivos reales para validar la capacidad de respuesta.

Consideraciones de iOS

Multitarea

La multitarea está muy controlada en iOS y hay una serie de reglas y comportamientos que la aplicación debe cumplir cuando otra aplicación está en primer plano, de lo contrario, iOS cancelará la aplicación.

Recursos específicos del dispositivo

En un factor de forma determinado, el hardware puede variar considerablemente entre diferentes modelos. Por ejemplo, algunos dispositivos tienen una cámara trasera, algunos también tienen una cámara frontal y otros no tienen ninguna.

Algunos dispositivos antiguos (iPhone 3G y anteriores) ni siquiera permiten la multitarea.

Debido a estas diferencias entre modelos de dispositivos, es importante comprobar la presencia de una característica antes de intentar usarla.

Restricciones específicas del sistema operativo

Para asegurarse de que las aplicaciones respondan correctamente y sean seguras, iOS impone una serie de reglas que deben cumplir las aplicaciones. Además de las reglas con respecto a la multitarea, hay una serie de métodos de evento de los que la aplicación debe volver en una determinada cantidad de tiempo, de lo contrario, iOS la cancelará.

También cabe observar que las aplicaciones se ejecutan en lo que se conoce como espacio aislado, un entorno que aplica restricciones de seguridad que restringen a qué puede tener acceso la aplicación. Por ejemplo, una aplicación puede leer y escribir en su propio directorio, pero si intenta escribir en otro directorio de aplicaciones, se cancelará.

Consideraciones de Android

Multitarea

La multitarea en Android tiene dos componentes; el primero es el ciclo de vida de la actividad. Cada pantalla de una aplicación de Android se representa mediante una actividad y hay un conjunto específico de eventos que se producen cuando una aplicación se coloca en segundo plano o vuelve al primer plano. Las aplicaciones deben respetar este ciclo de vida para responder y comportarse correctamente. Para obtener más información, consulte la guía [Activity Lifecycle](#) (Ciclo de vida de la actividad).

El segundo componente de la multitarea de Android es el uso de servicios. Los servicios son procesos de ejecución prolongada que existen independientes de una aplicación y se usan para ejecutar procesos mientras la aplicación está en segundo plano. Para obtener más información, consulte la guía [Creating Services](#) (Crear servicios).

Muchos dispositivos y muchos factores de forma

A diferencia de iOS, que tiene un conjunto pequeño de dispositivos, o incluso Windows Phone, que solo se ejecuta en dispositivos aprobados que cumplen un conjunto mínimo de requisitos de plataforma, Google no impone ningún límite sobre en qué dispositivos se puede ejecutar el sistema operativo Android. Este paradigma abierto da como resultado un entorno de productos lleno de una gran variedad de dispositivos diferentes con hardware, resoluciones y relaciones de pantalla, características del dispositivo y capacidades muy diferentes.

Debido a la fragmentación extrema de los dispositivos Android, la mayoría de los usuarios elige los 5 o 6 dispositivos más populares para los que diseñar y probar y les da prioridad a estos.

Consideraciones de seguridad

Todas las aplicaciones en el sistema operativo Android se ejecutan bajo una identidad distinta y aislada con permisos limitados. De manera predeterminada, las aplicaciones pueden hacer muy poco. Por ejemplo, sin permisos especiales, una aplicación no puede enviar un mensaje de texto, determinar el estado del teléfono, ni siquiera obtener acceso a Internet. Para tener acceso a estas características, las aplicaciones deben especificar en su archivo de manifiesto de la aplicación qué permisos quieren y cuándo se instalan. El

sistema operativo lee los permisos, notifica al usuario que la aplicación pide esos permisos y, después, permite al usuario continuar o cancelar la instalación. Este es un paso esencial en el modelo de distribución de Android, debido al modelo de tienda de aplicaciones abierto, ya que las aplicaciones no se mantienen de la misma forma que en iOS, por ejemplo. Para obtener una lista de permisos de aplicaciones, consulte el artículo de referencia [Manifest Permissions](#) (Permisos de manifiesto) en la documentación de Android.

Consideraciones sobre Windows Phone

Multitarea

La multitarea en Windows Phone también tiene dos partes: el ciclo de vida de páginas y aplicaciones y los procesos en segundo plano. Cada pantalla de una aplicación es una instancia de una clase de página, que tiene eventos asociados que se ponen en estado activo o inactivo (con reglas especiales para controlar el estado inactivo o "con marcador de exclusión"). Para obtener más información, consulte la documentación [Execution Model Overview for Windows Phone](#) (Introducción al modelo de ejecución de Windows Phone).

La segunda parte consiste en proporcionar agentes en segundo plano para tareas de procesamiento, incluso cuando la aplicación no se está ejecutando en primer plano. Puede encontrar más información sobre la programación de tareas periódicas o la creación de tareas en segundo plano con uso intensivo de recursos en [Background Agents Overview](#) (Introducción a los agentes en segundo plano).

Capacidades del DISPOSITIVO

Aunque el hardware de Windows Phone es bastante homogéneo debido a las instrucciones estrictas que proporciona Microsoft, hay componentes que son opcionales y, por tanto, requieren consideraciones especiales durante la codificación. Entre las capacidades de hardware opcionales se incluyen la cámara, la brújula y el giroscopio. También hay una clase especial de memoria insuficiente (256 MB) que requiere una consideración especial, o los desarrolladores pueden no participar en la compatibilidad con memoria insuficiente.

Base de datos

IOS y Android incluyen el motor de base de datos SQLite que permite el almacenamiento sofisticado de datos que también es multiplataforma. Windows Phone 7 no incluía una base de datos, mientras que Windows Phone 7.1 y 8 incluyen un [motor de base de datos local](#) que solo se puede consultar con [LINQ to SQL](#) y no es compatible con las consultas de Transact-SQL. Hay un [puerto de código abierto de SQLite](#) disponible que se puede agregar a las aplicaciones de Windows Phone para proporcionar la compatibilidad multiplataforma conocida de Transact-SQL.

Consideraciones de seguridad

Las aplicaciones de Windows Phone se ejecutan con un conjunto restringido de permisos que las aísla unas de otras y limita las operaciones que pueden realizar. El acceso a redes debe realizarse a través de las API específicas y la comunicación entre aplicaciones solo se puede realizar a través de mecanismos controlados. El acceso al sistema de archivos también está restringido; la API de almacenamiento aislado proporciona almacenamiento de par clave-valor y la capacidad de crear archivos y carpetas de un modo controlado (consulte [Isolated Storage Overview](#) (Introducción al almacenamiento aislado) para obtener más información).

El acceso de una aplicación a las características de hardware y sistema operativo se controla mediante las funcionalidades enumeradas en su archivo de manifiesto (similar a Android). El manifiesto debe declarar las características que requiere la aplicación para que los usuarios puedan ver y aceptar esos permisos y también para que el sistema operativo permita el acceso a las API. Las aplicaciones deben requerir acceso a características como los datos de contactos o citas, la cámara, la ubicación, la biblioteca multimedia, etc. Consulte la documentación [Application Manifest File](#) (Archivo de manifiesto de aplicaciones) de Microsoft para obtener información adicional.

Resumen

En esta guía, se le ha proporcionado una introducción al SDLC en lo referente al desarrollo móvil. Se han introducido consideraciones generales para la creación de aplicaciones

móviles y se ha examinado una serie de cuestiones específicas de la plataforma, incluidos el diseño, las prueba y la implementación.